

黑板

C \ C++

指向 $\frac{1}{2}$

*p = NULL;

char *p = NULL $\xrightarrow{\text{const int 0}}$
for i

C++11:

char *p = nullptr;

宏: #define const int MAXN=1w;

for 环定义: (for 环宏定义)

for(int i=0; i<=n; i++)

#define _for(i, a, b) for(int i=(a); i<(b); i++)

_for(i, 0, n) $\leftarrow$

vector<int> a

_for(i, 1, a.size()) { }

#define _rep(i, a, b) for(int i=(a); i<=(b); i++)

_rep(1, 0, n)

UVA1149

是对 N 个物件排序 大 $\rightarrow$ 小

然后依次决策

($\frac{1}{2}$) $\checkmark$ $\frac{1}{2}$

① 在没存放物品中, 考虑最轻的 i

它应该和谁(那个 j)放在一起, 选择最轻 j

它应该和谁(那个j)放在一起, 选择最轻j

i+j 放在一起

(if) 若 i+j 不能放在一起, 则 i 单独放

else i+j 放在一起

② 在考虑 i 时, 但是多个 j 可以和 i 放在一起

那么最轻的 j 是否可以和 i 放在一起

整数输入

```
int readint()
```

```
{ int x;
```

```
scanf("%d", &x); // cin >> x;
```

```
return x;
```

```
}
```

⑤ 排序

① 重载, 手写一个重载函数

② greater() 从大到小排序

③ less() 从小到大排序

sort(a, a+n, cmp)

容器 in for 循环

vector<int> vec;

✓ for (vector<int>::iterator it = vec.begin(); it != vec.end(); it++)
{ *it += 2;
cout << *it << endl;
}

C++11:

① for (const auto& p : vec)
cout << p << endl;

② for (auto& p : vec)

遍历

写法
[1]
注意
是

```
③ int arr[] = {1, 2, 3, 4, 5};  
for (int& x: arr) x += 2;
```

对容器 map, string, list, deque

类型推导 (auto)

如果使用比较长的声明，一般常见枚举器
(迭代器)
[指针]

```
vector<int> vec;
```

```
vector<int>::iterator cit = vec.begin();
```

```
auto cit = vec.begin();
```

编译器 g++ (从 c++11 起) 遇到 auto 之后
就会根据右边的表达式自动推导出其数据类型，
同时也支持引用 & 类的变量

```
vector<int> vec = {1, 2, 3};
```

```
auto& v2 = vec[1];
```

```
v2 += 3; // vec 就变成了 {1, 5, 3}
```